

# DP Computer Science: A3.2.4 Constructing Tables

---

## 1. Core Concepts & Learning Objectives

### Learning Intentions

By the end of this lesson, students will be able to:

- **Construct tables** for relational databases
  - Explain the **relationship between tables** using **primary keys**, **foreign keys**, **composite keys**, and **concatenated keys**
  - Discuss the importance of **well-defined tables** in ensuring **data integrity**
- 

### Key Vocabulary

| Term                  | Definition                                                                                              |
|-----------------------|---------------------------------------------------------------------------------------------------------|
| Table                 | A collection of related data organised in rows and columns                                              |
| Primary key           | A unique identifier for each record in a table; ensures no duplicate rows                               |
| Foreign key           | A field in one table that refers to the primary key of another table; establishes relationships         |
| Composite key         | A primary key made up of two or more columns                                                            |
| Concatenated key      | A key formed by combining two or more attributes (similar to composite key; often used interchangeably) |
| Data integrity        | The accuracy, consistency, and reliability of data in a database                                        |
| Referential integrity | Ensures that foreign key values in one table match primary key values in another table                  |

| Term          | Definition                                                                |
|---------------|---------------------------------------------------------------------------|
| Normalisation | The process of organising data to reduce redundancy and improve integrity |

## IB ATL Skills

| Skill           | Description                                                                                                              |
|-----------------|--------------------------------------------------------------------------------------------------------------------------|
| Thinking        | Critical thinking (analysing table structures, key relationships); application (designing tables for specific scenarios) |
| Communication   | Using correct terminology; explaining the role of keys in table relationships                                            |
| Self-Management | Organisation (structuring data effectively in tables)                                                                    |

## 2. Constructing Tables

### What is a Table?

A table is a collection of related data organised in rows and columns. Each row (record) represents a single instance, and each column (field) represents an attribute.

### Table Structure

text

TABLE STRUCTURE

STUDENTS

StudentID | FirstName | LastName | DateOfBirth | Age |

|                                                   |         |       |            |    |  |  |
|---------------------------------------------------|---------|-------|------------|----|--|--|
|                                                   | (PK)    |       |            |    |  |  |
| 101                                               | Alice   | Smith | 2008-03-15 | 16 |  |  |
| 102                                               | Bob     | Jones | 2007-07-22 | 17 |  |  |
| 103                                               | Charlie | Brown | 2009-01-10 | 15 |  |  |
| Columns = Attributes (e.g., StudentID, FirstName) |         |       |            |    |  |  |
| Rows = Records (e.g., Alice Smith)                |         |       |            |    |  |  |

## Guidelines for Constructing Tables

| Guideline                     | Description                                                              |
|-------------------------------|--------------------------------------------------------------------------|
| Choose appropriate data types | Use the correct data type for each column (INTEGER, VARCHAR, DATE, etc.) |
| Define a primary key          | Every table should have a unique identifier                              |
| Avoid redundancy              | Store data in one place only (normalisation)                             |
| Use meaningful names          | Table and column names should be descriptive                             |
| Enforce constraints           | Use NOT NULL, UNIQUE, and CHECK constraints where appropriate            |
| Plan relationships            | Consider how tables will relate to each other                            |

## 3. Types of Keys

### Primary Key

A primary key is a unique identifier for each record in a table. It ensures that no two rows are identical and allows each record to be uniquely referenced.

| Aspect          | Description                     |
|-----------------|---------------------------------|
| Purpose         | Uniquely identifies each record |
| Characteristics | Unique, NOT NULL, immutable     |
| Example         | StudentID in a Students table   |

text

PRIMARY KEY EXAMPLE

STUDENTS

| StudentID (PK) | FirstName | LastName | DateOfBirth |
|----------------|-----------|----------|-------------|
| 101            | Alice     | Smith    | 2008-03-15  |
| 102            | Bob       | Jones    | 2007-07-22  |
| 103            | Charlie   | Brown    | 2009-01-10  |

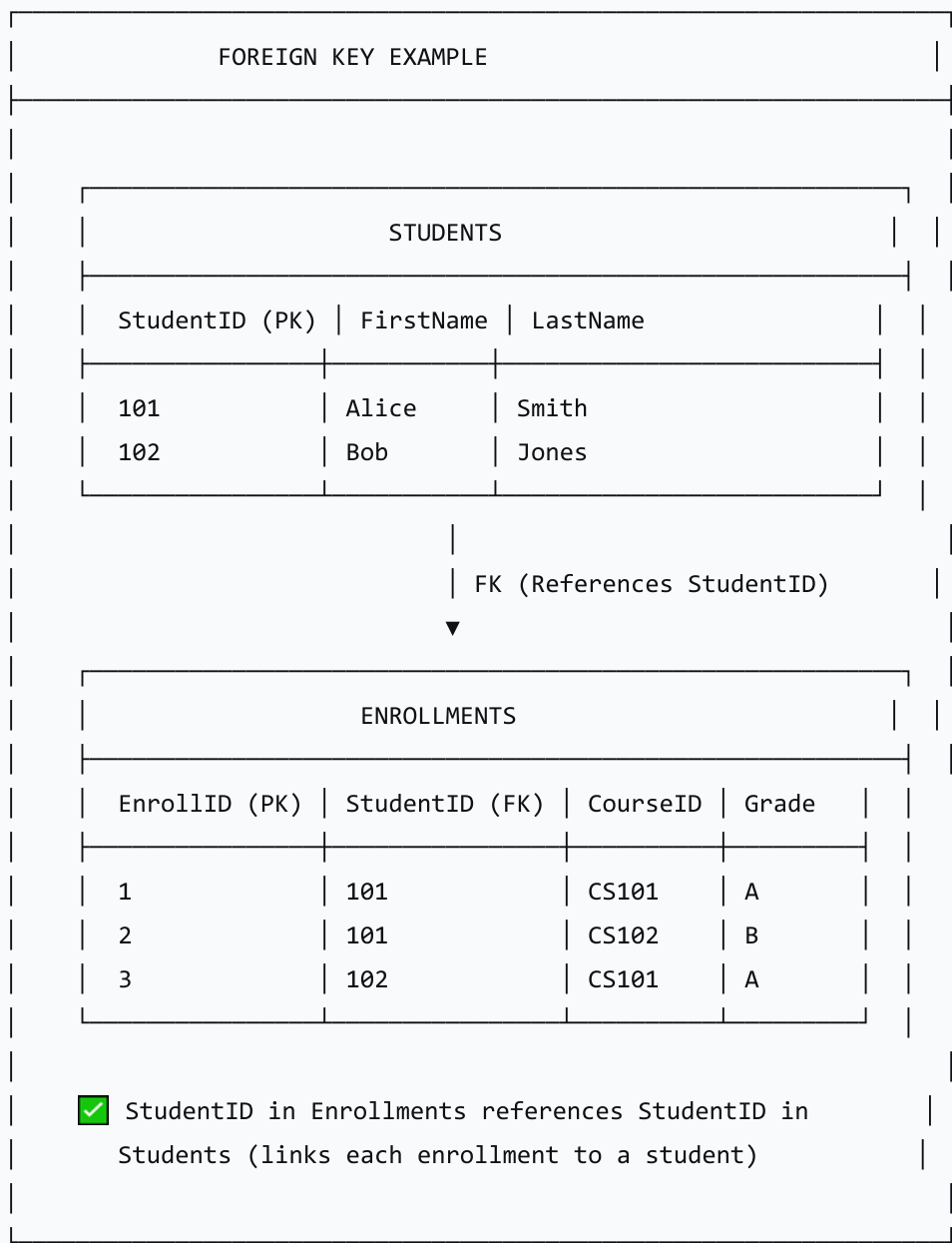
 StudentID uniquely identifies each student

## Foreign Key

A foreign key is a field in one table that refers to the primary key of another table. It establishes a relationship between tables.

| Aspect          | Description                                                        |
|-----------------|--------------------------------------------------------------------|
| Purpose         | Links tables together; establishes relationships                   |
| Characteristics | References a primary key; can have NULL values                     |
| Example         | StudentID in an Enrollments table (referencing Students.StudentID) |

text



## Composite Key

A composite key is a primary key made up of two or more columns. It is used when a single column cannot uniquely identify a record.

| Aspect          | Description                                        |
|-----------------|----------------------------------------------------|
| Purpose         | Creates a unique identifier using multiple columns |
| Characteristics | Made of 2+ columns; the combination must be unique |

| Aspect  | Description                                   |
|---------|-----------------------------------------------|
| Example | (StudentID, CourseID) in an Enrollments table |

text

COMPOSITE KEY EXAMPLE

ENROLLMENTS

(Composite Key: StudentID + CourseID)

| StudentID (PK) | CourseID (PK) | Grade | Semester    |
|----------------|---------------|-------|-------------|
| 101            | CS101         | A     | Fall 2025   |
| 101            | CS102         | B     | Fall 2025   |
| 102            | CS101         | A     | Spring 2026 |

✓

The combination (StudentID, CourseID) uniquely identifies each record

## Concatenated Key

A concatenated key is formed by combining two or more attributes into a single key. It is similar to a composite key and the terms are often used interchangeably.

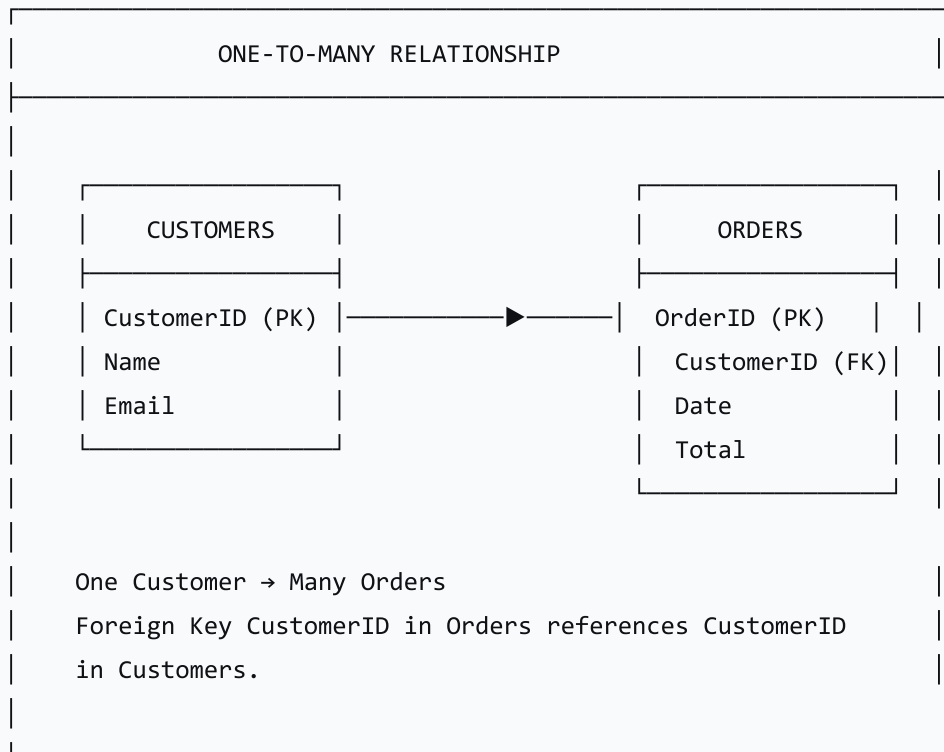
| Aspect          | Description                                         |
|-----------------|-----------------------------------------------------|
| Purpose         | Creates a unique identifier by concatenating fields |
| Characteristics | Often a string formed by joining fields             |
| Example         | StudentID + CourseID combined as "101CS101"         |

**Note:** In most practical contexts, "composite key" and "concatenated key" are used synonymously to refer to a key made up of multiple columns.

## 4. Relationships Between Tables

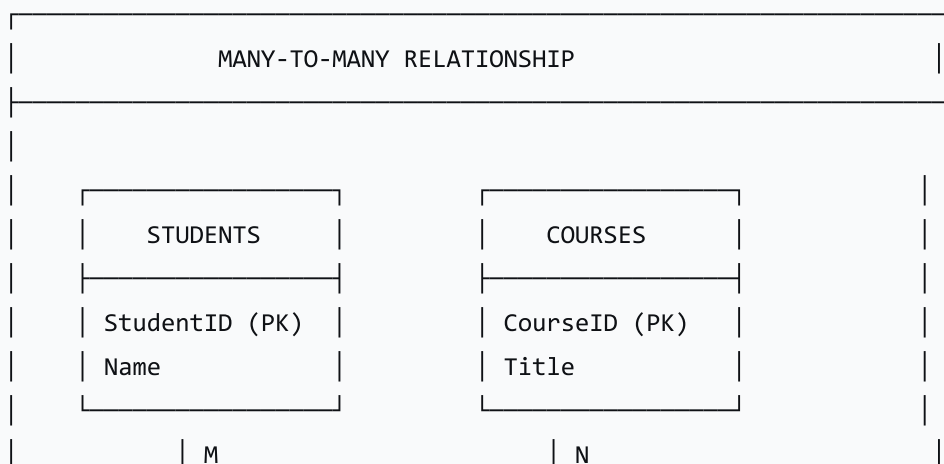
### One-to-Many Relationship (1:N)

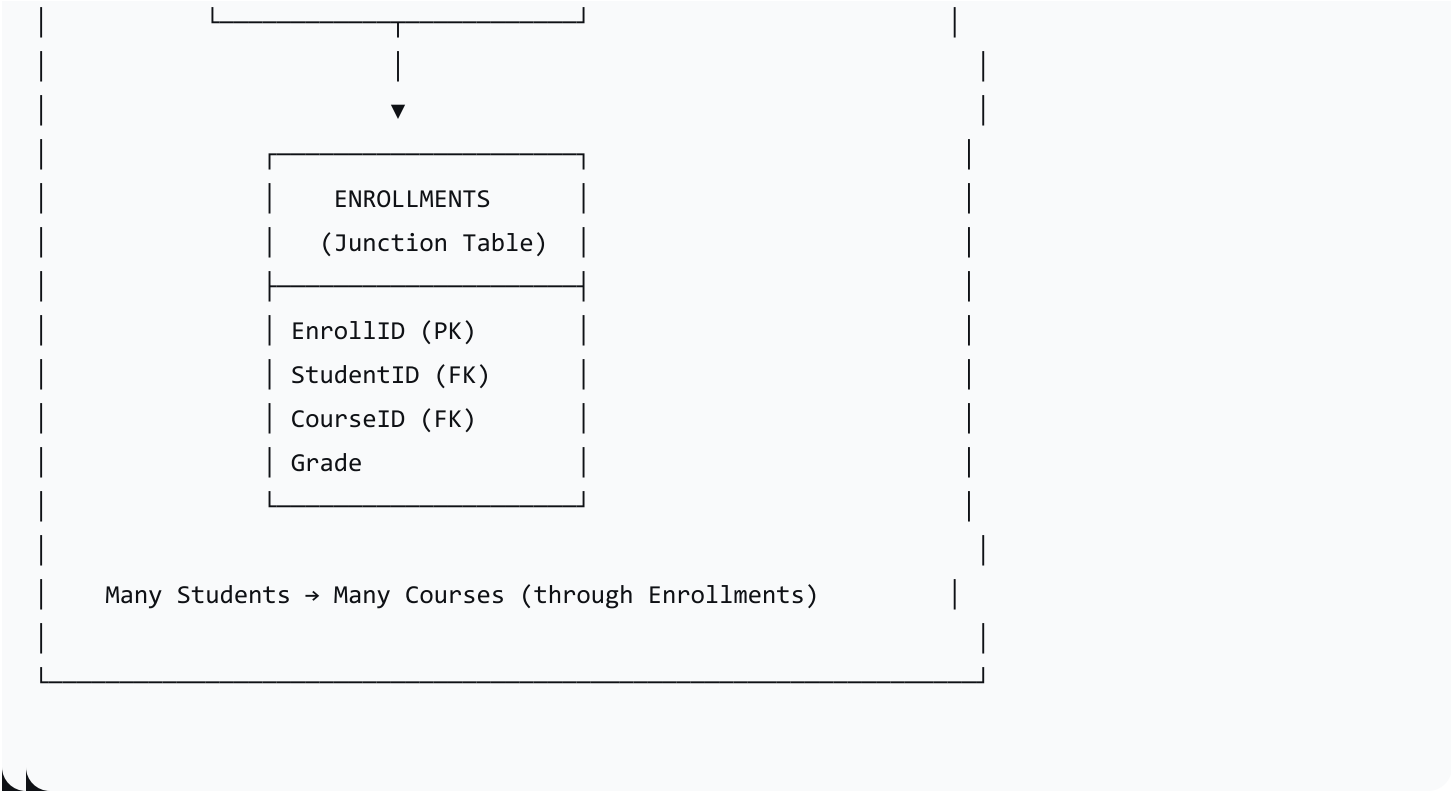
text



### Many-to-Many Relationship (M:N)

text





## 5. Importance of Well-Defined Tables

### Why Well-Defined Tables Matter

| Reason              | Description                                       |
|---------------------|---------------------------------------------------|
| Data Integrity      | Prevents data inconsistencies and errors          |
| Efficient Retrieval | Well-structured tables enable fast queries        |
| Reduced Redundancy  | Prevents duplicate data                           |
| Easier Maintenance  | Simple to update and modify                       |
| Clear Relationships | Relationships are explicit and easy to understand |
| Scalability         | Database can grow without issues                  |

### Common Problems with Poorly Designed Tables

| Problem              | Example                                     | Consequence                                 |
|----------------------|---------------------------------------------|---------------------------------------------|
| Missing Primary Key  | No unique identifier                        | Duplicate records; cannot reference rows    |
| Data Redundancy      | Customer details repeated in every order    | Wasted space; update anomalies              |
| No Foreign Key       | Orders table has no CustomerID              | Cannot link orders to customers             |
| Incorrect Data Types | Age stored as VARCHAR                       | Sorting errors; cannot perform calculations |
| No Constraints       | NULL values allowed where they shouldn't be | Incomplete data; integrity issues           |

## 6. Activity: Poorly Designed Table Analysis

### Instructions

- 1. In groups, analyse the poorly designed table
- 2. Identify the problems
- 3. Propose improvements

### Scenario: Student Records

text

|                       |     |         |         |         |         |  |  |
|-----------------------|-----|---------|---------|---------|---------|--|--|
| POORLY DESIGNED TABLE |     |         |         |         |         |  |  |
| STUDENT_RECORDS       |     |         |         |         |         |  |  |
| Name                  | Age | Address | Course1 | Course2 | Course3 |  |  |
| Alice                 | 16  | 1 Main  | CS101   | CS102   | NULL    |  |  |

|  |       |    |        |       |      |      |  |  |
|--|-------|----|--------|-------|------|------|--|--|
|  | Bob   | 17 | 2 Oak  | CS101 | NULL | NULL |  |  |
|  | Alice | 16 | 1 Main | CS103 | NULL | NULL |  |  |
|  |       |    |        |       |      |      |  |  |

## Problems Identified

| Problem            | Description                                   |
|--------------------|-----------------------------------------------|
| No Primary Key     | Cannot uniquely identify records              |
| Duplicate Data     | Alice appears twice; her address is repeated  |
| Inflexible Courses | Fixed number of course columns (limited to 3) |
| Poor Data Type     | Age stored as text? (implied by VARCHAR)      |

## Proposed Solution

text

|                 |       |     |           |  |
|-----------------|-------|-----|-----------|--|
| IMPROVED DESIGN |       |     |           |  |
| STUDENTS        |       |     |           |  |
| StudentID (PK)  | Name  | Age | Address   |  |
| 101             | Alice | 16  | 1 Main St |  |
| 102             | Bob   | 17  | 2 Oak Ave |  |
| COURSES         |       |     |           |  |
| CourseID (PK)   | Title |     |           |  |

|  |               |                    |               |  |
|--|---------------|--------------------|---------------|--|
|  | CS101         | Introduction to CS |               |  |
|  | CS102         | Data Structures    |               |  |
|  | CS103         | Databases          |               |  |
|  |               |                    |               |  |
|  | ENROLLMENTS   |                    |               |  |
|  | EnrollID (PK) | StudentID (FK)     | CourseID (FK) |  |
|  | 1             | 101                | CS101         |  |
|  | 2             | 101                | CS102         |  |
|  | 3             | 102                | CS101         |  |
|  | 4             | 101                | CS103         |  |

## 7. Lesson Sequence

### Lesson Plan (60 minutes)

| Phase                | Time   | Activity                                       |
|----------------------|--------|------------------------------------------------|
| Introduction         | 5 min  | Review relational databases and tables         |
| Constructing Tables  | 15 min | Table components, guidelines, visual examples  |
| Keys & Relationships | 20 min | Primary, foreign, composite keys; diagrams     |
| Activity             | 15 min | Poorly designed table analysis and improvement |
| Wrap-up              | 5 min  | Summary; Q&A                                   |

## 8. Assessment Activities

| Assessment Type     | Description                                                  |
|---------------------|--------------------------------------------------------------|
| Class Participation | Observe engagement and contributions                         |
| Activity            | Assess ability to identify problems and improve table design |
| Exit Ticket         | Gauge understanding and identify areas needing clarification |

### Sample Exit Ticket Questions:

1. What is a primary key?
2. What is a foreign key and why is it important?
3. When would you use a composite key?
4. Why are well-defined tables important for data integrity?

## 9. Differentiation

| Level     | Strategy                                                       |
|-----------|----------------------------------------------------------------|
| Support   | Provide visual aids and scaffolded examples                    |
| Challenge | Design a complete database schema with multiple related tables |

## 10. Resources

- **Presentation** (Slide Deck)
- **eBook** (A3.2.4 with examples)
- **Worksheet:** Table construction and key identification
- **Quizlet** (A3.2.4 vocabulary flashcards)

## 11. Summary: Key Takeaways

| Concept             | Key Point                                      |
|---------------------|------------------------------------------------|
| Table               | Organises data into rows and columns           |
| Primary Key         | Unique identifier for each record              |
| Foreign Key         | Links tables; references a primary key         |
| Composite Key       | Primary key made of multiple columns           |
| Data Integrity      | Ensures accuracy, consistency, and reliability |
| Well-Defined Tables | Prevent redundancy, support efficient queries  |

text

KEY REMINDERS

✓

 Every table needs a primary key

✓

 Foreign keys establish relationships

✓

 Composite keys use multiple columns

✓

 Well-designed tables ensure data integrity

✓

 Avoid redundancy and inflexibility

"Well-defined tables are the foundation of a reliable database—they ensure data integrity, reduce redundancy, and make relationships clear."